The Minimum Sufficient Code Context Problem

Complexity, Discovery Overhead, and Approximation in Coding Agents

Michael Welsch GPT-5.6 Sol Fable 5.0

July 19, 2026

Abstract

Repository-level coding couples two problems: discovering the code required by a task and solving the task from the discovered context. We formalize the discovery target as the minimum sufficient code context (MSCC): the token-minimal context from which a fixed agent reaches a fixed success threshold without further repository access. A graph-grounded decision variant of MSCC construction is NP-complete (minimum-weight directed Steiner, by reduction from Set Cover), so general MSCC optimization is NP-hard, and we prove lower bounds for the two information losses of lexical navigation: a behavioral query does not order lexically indistinguishable entry signatures, and an untyped text occurrence does not identify a resolved program edge. A combined no-trade-off theorem (formally, a dominance statement) then states when a policy with semantic entry, typed expansion, exact-source validation, and lexical fallback is strictly cheaper and correctness-noninferior than a lexical agent – conditional on measurable retrieval, validation, and error terms, which our experiments estimate rather than assume.

Greppy is the evaluated framework: it retains byte-exact `grep` and file reads and adds semantic signature retrieval, a Tree-sitter-derived signature graph, and bounded exact source expansion. A versioned 2×2 experiment on repository question answering and localization (not end-to-end edit success) separates tool-surface and instruction effects across pinned repositories and coding models, followed by mechanism ablations, a structured-search baseline, and a direct measurement of the distance between agent trajectories and the exact MSCC on an exhaustively solvable task family. The theory predicts that the advantage grows with measured lexical ambiguity, independently of coding-model strength; the experiment reports the terms needed to test exactly that. Across four coding models spanning three providers (MiniMax-M3, GLM-5.2, Qwen3.6-27B, and Kimi-K3), the structured surface dominates the lexical baseline at every call budget: it answers 6 to 50 percentage points more tasks correctly at matched budget (on MiniMax-M3, 2.4 times as many at the smallest budget, 79.0% versus 33.1%), and reaches the lexical agent's best natural-stop correctness at 37–80% lower billed cost. The factorial design isolates the source of the effect: the MSCC instruction, added to either the lexical or the structured surface, produces no significant correctness change in any of the four panels and only adds cost (up to $21 per 1,000 tasks on Kimi-K3) – the advantage is carried by the tool surface, not by prompt engineering. Outside the navigation-heavy regime, the artifact's release-gated paired-edit benchmark shows the same surface to be cost- and correctness-noninferior in billed provider dollars. Adopting the structured surface therefore involves no trade-off at any measured operating point.

1 Problem modelling

A repository task can be divided conceptually into two steps. First, a retrieval operation places all code needed for the task in the model context. Second, the model answers the question or emits the edit in one attempt, without further repository access. If the first step returned the MSCC, it would supply enough code and no avoidable code. For the formal problem class defined below, no polynomial-time operation can guarantee that result in the worst case unless P = NP.

Coding agents implement a practical, uncertified approximation of the first step. Here *approximation* means a policy that seeks a sufficient context; no multiplicative or additive approximation ratio is claimed. Agents alternate queries, source reads, and intermediate decisions until they either stop with a sufficient context or answer without one. Returned tool results add input to the transcript; search decisions and tool calls add decoded output; both remain available through the model's key–value state. These discovery costs are incurred before the final answer and are distinct from the code that makes the answer possible.

Terminology. In this paper, *lexical baseline* means a complete coding-agent action surface containing `grep`,[1] file reads, ordinary shell navigation, and the final answer; it is not synonymous with the single command. Greppy is simply the framework name chosen because the implementation preserves this `grep` path and adds optional repository navigation operations; the name is descriptive rather than an acronym. The added operations provide semantic signature entry, approximate signature relations, and bounded exact-source retrieval; the coding model still chooses every call.

Two uncertainties are distinct. The *entry problem* is to locate a first relevant definition from the task description. The *expansion problem* is to find the callers, callees, types, imports, tests, and configuration that must be inspected around that entry. A policy restricted to `grep` and file reads addresses either problem indirectly through text occurrences. Semantic similarity targets entry; a signature graph targets expansion; exact source remains the evidence needed to solve.

[1] The Unix command name comes from the `ed` form `g/re/p`: apply a regular expression globally and print the matching lines [5]. Operationally, `grep` selects file lines matching literal or regular-expression patterns.

The paper contributes a graph formulation of MSCC, an NP-completeness proof, lexical inspection lower bounds for entry and expansion, a dominance theorem for the combined Greppy–MSCC policy, and an experiment that measures both end-to-end cost–correctness and distance from an exactly enumerated MSCC, separating tool-surface from instruction effects: the structured tool surface realizes the MSCC logic and carries the entire advantage, while the MSCC instruction added to either surface is a null control that changes no panel's correctness and only adds cost.

1.1 Relation to prior work

The closest context-minimization work is Oracle-guided Code Distillation (OCD), which searches for a 1-minimal subsequence of an initially supplied repair context and uses those contexts to train SWEzze [9]. A 1-minimal result cannot lose any single retained element without losing the tested repair, but need not be the globally shortest context. MSCC instead minimizes token length over a fixed candidate family and makes the transcript required to find that context part of the analysis. Both notions are relative to a task, model, and success test; MSCC is additionally relative to tokenizer and serialization. Joren et al. use *sufficient context* to separate retrieval from answer-generation failure in general RAG systems [10]; MSCC adds the global minimization and discovery-cost questions.

Program slicing asks for statements relevant to a program point or behavior [20]; delta debugging and syntax-guided reduction minimize failure-inducing inputs or programs under an executable property test [16, 19]. An MSCC need not be an executable program or a semantics-preserving slice. It may contain source and navigation metadata, and its order and serialization can matter to the coding model. Sufficiency is not necessarily monotone because additional plausible code can distract the model. These differences prevent direct transfer of classical slicing guarantees, while the shared minimization structure motivates the restricted hardness result below.

Sequential repository discovery is also a metareasoning problem: the agent spends computation to decide which observation to buy next [17]. Adaptive submodular coverage provides greedy guarantees under monotonicity and diminishing returns [4, 6]. Code context need not satisfy those conditions: two definitions may be useless separately but sufficient together. We therefore make no general greedy approximation claim; the lower bounds below concern explicit repository families.

Repository-level retrieval has progressed from iterative similarity retrieval in RepoCoder [23] and selective retrieval in Repoformer [21] to dataflow-guided context in DraCo [3]. CodexGraph exposes a repository graph database to an LLM [15]; LocAgent uses a heterogeneous graph for localization [2]. FastCode is especially close to the present mechanism: it scouts a semantic–structural repository map before loading full source [13]. Recent systems combine signals more directly: CodeRAG retrieves relevant and necessary knowledge [26], AIRCoder adapts among retrieval dimensions [18], RepoShapley filters context by estimated contribution [8], and LARGER expands a repository graph from lexical anchors [7]. These systems establish strong empirical baselines for structured retrieval. Their reported accuracy does not determine whether an added retrieval surface is cost-improving after failures, exact-source validation, LLM reasoning, and index construction are counted.

Agentless separates localization from repair [22]. ContextBench measures trajectory-level context recall, precision, and efficiency [12]; SWE-Explore isolates ranked code-region exploration under a line budget [25]; CORE-Bench spans code understanding, issue localization, and broader-context retrieval [24]. Long-context work further shows that supplying more text does not guarantee that the model uses it effectively [14]. These findings motivate an operational frontier over end-to-end correctness and discovery cost, together with mechanism-specific measurements for semantic entry and structural expansion.

1.2 Repository graph and sufficient context

1.2.1 A source-labelled repository graph

Fix a repository snapshot R. A *signature record* is an addressable declaration represented by language, path, source span, declaration kind, qualified name, and those parameter or receiver types that the chosen front end resolves statically. This record is an index key, not a claim that every language has Java-style overload signatures. Let V be the finite set of such records for functions, methods, types, modules, and other addressable definitions. Each $v \in V$ has an exact source payload $\mathrm{src}(v)$. Let the reference static graph $G_R = (V, E)$ be defined, for a fixed build configuration and relation schema, by the language front end's name and type resolution. Greppy does not compute G_R; it computes the Tree-sitter-based approximation

$$\widehat{G}_R = (V, \widehat{E}), \qquad \widehat{E} = \bigcup_{r \in \mathcal{R}} \widehat{E}_r, \tag{1}$$

where $\mathcal{R}$ includes typed relations such as Calls, Uses, TypeRef, Imports, and containment. The hat is essential. Even a compiler-defined graph is a static target rather than a complete runtime execution graph; dynamic dispatch, reflection, generated code, macros, and runtime injection may require stronger analysis. Tree-sitter adds a second approximation layer because it does not reproduce complete compiler name and type resolution. For task τ, let G_τ^{cmp} be the oracle-labelled statement that every relation required by the task oracle is present in $\widehat{G}_R$. This completeness event is not observable merely by reading returned neighbors: exact source can support a returned edge, but cannot prove that no required edge was omitted. For a frozen manifest it has the deterministic indicator $g_\tau = \mathbf{1}\{G_\tau^{\mathrm{cmp}}\}$. Over a prespecified task measure $\mathcal{D}$, graph coverage is $\gamma_{\mathcal{D}} = \mathbb{E}_{\tau \sim \mathcal{D}}[g_\tau]$; the finite study uses the equal-repository-weighted empirical analogue. Thus coverage is an evaluation quantity, never a runtime probability inferred from returned neighbors.

Representation lemma. Every finite, successfully resolved repository snapshot has a finite source-labelled directed multigraph that represents exactly any fixed finite set of compiler-resolved static relations. Create one vertex for every resolved declaration and attach its exact source payload. For every resolved relation instance (u, r, v), add an edge labelled r from u to v; containment edges attach declarations to files and modules. Finiteness of the snapshot and

relation schema makes both vertex and edge sets finite. The construction is exact for the chosen static semantics because edge membership is defined by that resolver. It says nothing about runtime relations outside those semantics.

The construction identifies the static object being approximated; it does not make Tree-sitter equivalent to the reference resolver. Comparing $\widehat{G}_R$ with G_R on a compiler-resolved corpus measures missed and false edges; runtime behavior requires a different reference. The source labels are also indispensable: removing them would make the representation lossy for coding tasks.

1.2.2 Minimum sufficient code context

Fix a repository snapshot R, task τ, coding agent A, verifier W, success threshold θ, tokenizer T, serialization σ, and finite candidate family $\mathcal{H}_R$. Each $H \in \mathcal{H}_R$ is a sequence of exact source payloads and records from a finite metadata universe fixed before the run; $\sigma(H)$ is the string presented to the model and $\mathrm{tok}(H) = |T(\sigma(H))|$. The agent receives only τ and H and must emit its final answer or edit without further repository access. For model randomness ξ, define

$$Q(H; R, \tau, A, W) = \mathbb{P}_\xi \left[W(R, \tau, A_\xi(\tau, H)) = 1 \right]. \quad (2)$$

The minimum sufficient code context is

$$H^* \in \arg\min_{H \in \mathcal{H}_R} \mathrm{tok}(H) \quad \text{s.t.} \quad Q(H; R, \tau, A, W) \geq \theta. \quad (3)$$

If the sufficient subset of $\mathcal{H}_R$ is nonempty, an MSCC exists: token length maps that finite subset to a nonempty finite subset of $\mathbb{N}$, which has a least element. The minimizer need not be unique. If no rendered context reaches θ, the MSCC is undefined for the fixed tuple $(R, \tau, A, W, \theta, T, \sigma, \mathcal{H}_R)$. Thus an MSCC is not an intrinsic property of a repository or task. Changing the model, verifier, threshold, admissible metadata, tokenizer, order, or serialization can change both sufficiency and the minimum.

To expose the graph-theoretic core, add a task vertex s and a finite set of required fact vertices T_τ. A source-grounded context $U \subseteq V$ is *reachability-sufficient* when the subgraph induced by $U \cup \{s\} \cup T_\tau$ contains an s–t path for every $t \in T_\tau$. Source vertices have weight $w(v) = \mathrm{tok}(\mathrm{src}(v))$; task and fact vertices have zero weight. The corresponding restricted MSCC is the minimum-weight source-bearing directed Steiner subgraph

$$\min_{U \subseteq V} \sum_{v \in U} w(v) \quad \text{s.t.} \quad \forall t \in T_\tau : \; s \rightsquigarrow t \text{ in } G_R[U \cup \{s\} \cup T_\tau]. \quad (4)$$

Theorem 1 (Graph-grounded MSCC is NP-complete)**.** *The decision version of Eq. (4)—whether a reachability-sufficient source context of weight at most K exists—is NP-complete, even for a three-layer acyclic signature graph with unit-weight source vertices. Consequently, exact MSCC construction in Eq. (3) is NP-hard.*

In plain terms: there is no shortcut that always finds the smallest sufficient context; even with a perfect dependency graph, choosing the cheapest set of source blocks that covers everything a task needs is as hard as classic covering problems. Any practical navigator must therefore be a good heuristic, and its value is an empirical question – which is why this paper measures cost and correctness instead of claiming optimality.

Proof. Reduce Set Cover. Given universe $\mathcal{U} = \{e_1, \ldots, e_m\}$ and sets $S_1, \ldots, S_n$, construct a directed acyclic graph with task root s, one unit-weight source vertex v_i for each set S_i, and one zero-weight fact terminal t_j for each element e_j. Add every edge $s \to v_i$ and add $v_i \to t_j$ exactly when $e_j \in S_i$. This is a source-labelled signature graph: v_i may be realized by a function whose resolved references are the fact signatures t_j.

For any $J \subseteq \{1, \ldots, n\}$, the subgraph induced by $\{v_i : i \in J\} \cup \{s\} \cup T_\tau$ reaches every terminal if and only if $\bigcup_{i \in J} S_i = \mathcal{U}$. Its source weight is $|J|$. Hence it has weight at most K exactly when the Set Cover instance has a cover of size at most K. The construction is polynomial. Membership in NP follows because a vertex set is a polynomial certificate and all terminal reachability tests and the weight sum are computable in polynomial time. Set Cover is NP-complete [11]; therefore the restricted graph problem is NP-complete. Fix a deterministic agent that emits the terminals reachable from the source records in its context and a verifier that accepts exactly when all terminals are emitted. Reachability sufficiency is then the threshold-one predicate in Eq. (2). The graph problem is therefore a deterministic special case of Eq. (3), so the general optimization problem is NP-hard. □

This reduction has a direct operational meaning. A hypothetical tool that always returned H^* would solve a node-weighted directed Steiner problem before the coding model answered. Unless P = NP, no polynomial-time repository tool can guarantee that result for every instance. An agent must therefore approximate the source-bearing subgraph and pay for the observations and decoded decisions used to find it.

1.2.3 Discovery overhead

For a discovery transcript D, let I_D^{nav} be the non-model tokens added specifically for navigation before the final answer. This includes condition-specific instructions, tool descriptions, returned source, and non-source tool results; task text and instructions shared by all conditions are excluded. Let $E(D)$ be the ordered sequence of evidence records returned by tools, including exact source, paths, signatures, typed relations, and frozen indexed hints. Define $\mathcal{H}_R(D) \subseteq \mathcal{H}_R$ to contain exactly those candidate contexts whose canonical record sequences occur as order-preserving subsequences of $E(D)$. Free-form text not admitted to the candidate family can aid discovery but cannot become part of the post-hoc minimum. Let $H^\dagger(D)$ minimize token length over the sufficient members of $\mathcal{H}_R(D)$. We require the frozen serialization to be *transcript-compatible*: for every $H \in \mathcal{H}_R(D)$, its canonical token count is no greater than the navigation-input tokens containing those evidence records. Operationally, σ is the exact record payload projected from the tool message under the same tokenizer and message framing. When a suffi-

cient member exists, the discovery-input overhead above the MSCC decomposes as

$$\Omega_I(D) = \underbrace{\mathrm{tok}(H^\dagger) - \mathrm{tok}(H^*)}_{\text{larger sufficient context}} + \underbrace{I_D^{\mathrm{nav}} - \mathrm{tok}(H^\dagger)}_{\text{residual navigation input}} . \quad (5)$$

All model-decoded tokens used before the final answer—including search reasoning and tool-call arguments—form discovery-output overhead $\Omega_O(D) = O_D^{\mathrm{nav}}$. These quantities describe what is added to the transcript relative to receiving H^* directly; they do not charge previous model output a second time as new input. If no sufficient subsequence of returned evidence exists, the run is a discovery failure rather than an overhead observation. Algebraically, Eq. (5) equals $I_D^{\mathrm{nav}} - \mathrm{tok}(H^*)$. Its two latent terms distinguish an over-complete but sufficient evidence set from material that contributes only to finding that set. They are identifiable only with additional context ablation or a controlled sufficiency predicate; ordinary end-to-end success identifies only the total. The oracle-minimal-context suite of Section 4.3 provides exactly this instrumentation: its enumerated verifier and the deletion ablation on larger repositories separate the two terms for the trajectories measured there. Because $H^\dagger \in \mathcal{H}_R$ is sufficient, global minimality of H^* guarantees $\mathrm{tok}(H^\dagger) - \mathrm{tok}(H^*) \geq 0$; transcript compatibility guarantees the second term is also nonnegative. Without that compatibility, the equality still holds algebraically but the residual term has no nonnegative-overhead interpretation. On graph-addressable tasks, $H^\dagger$ may contain source attached to a small connected or nearly connected subgraph; neither the graph nor an embedding replaces that source.

1.2.4 Discovery policies and total cost

Let $\mathcal{A}$ be a tool action set. A policy $\pi \in \Pi(\mathcal{A})$ maps the persistent transcript to the next tool call or a final answer. Its run produces correctness $Y_\pi \in \{0, 1\}$ and cost

$$C_\pi = \frac{B_\pi}{Q_{\mathrm{reuse}}} + p_I I_\pi + p_R R_\pi + p_W W_\pi + p_O O_\pi + \lambda_t t_\pi . \quad (6)$$

B_π is the one-time local index-construction cost in the same scalar unit, attributed over Q_{reuse} uses; I, R, W, O are uncached input, cache reads, cache writes, and decoded output; t is latency in seconds and λ_t converts it to that scalar unit. When local compute is not monetized, its time and storage are reported separately, $B_\pi = 0$ on the provider-dollar axis, and $\lambda_t t$ is evaluated as a sensitivity rather than added dimensionally. Reasoning and tool-call arguments belong to O. Tool results are appended input, while earlier output retained in the KV cache is not charged again as new input. The theorems require only that cost is nonnegative and that identical trajectories have identical cost; the experiment preserves the components rather than hiding them behind one token sum.

Q_{reuse} counts deployment task attempts to which the same frozen index artifact is made available after one build, whether or not a particular attempt selects a structured action. Lexical-only surfaces do not count, and benchmark repetitions are not treated as deployment reuse. Consequently an augmented run with $Z = 0$ can still carry its allocated build cost through $d_{\bar{Z}}$; the selected branch carries the same allocation through a.

For target correctness α, define the optimal achievable cost

$$C^*_{\mathcal{A}}(\alpha) = \inf_{\pi \in \Pi(\mathcal{A})} \{\mathbb{E}[C_\pi] : \mathbb{P}(Y_\pi = 1) \geq \alpha\} . \quad (7)$$

Equation (7) describes the behavior of the complete discovery procedure; H^* remains the task-level reference for input overhead.

2 The no-trade-off guarantee

2.1 Lexical and structured actions

Let $\mathcal{A}_0$ contain ordinary `grep`, file reads, and the final answer. A `grep` observation is lexical: for pattern p, it returns matching byte ranges and requested surrounding lines. An agent can implement sophisticated navigation with these primitives, but it must choose queries, interpret matches, open source, and decide when enough evidence has been seen.

Let the augmented set be

$$\mathcal{A}_1 = \mathcal{A}_0 \cup \{S_k(q), N_r(v), P(u, v), X(U)\}. \quad (8)$$

$S_k(q)$ returns the k signatures most similar to natural-language query q; $N_r(v)$ returns typed graph neighbors; $P(u, v)$ returns a graph path; and $X(U)$ returns bounded exact source for selected signatures. The coding model selects these operations and decides when to stop.

Greppy implements (8) in one binary. Crucially, the augmented agent surface retains the identical direct system `grep` action. Greppy's own ordinary invocations also execute the installed `grep` program and forward arguments, stdout, stderr, and exit status byte-for-byte without opening an index or mutating a cache. Structured commands activate the additional local index [1]. The passthrough supplies observation equivalence for the baseline action. It does not eliminate the schema, instruction, or dispatch cost of exposing the larger interface; those terms appear explicitly below.

Proposition 1 (Weak dominance of the Greppy action set)**.** *Let D_{int} be the mandatory cost of exposing the augmented action set. Because $\mathcal{A}_0 \subset \mathcal{A}_1$ and every baseline action has the same observation under both surfaces,*

$$C^*_{\mathcal{A}_1}(\alpha) \leq C^*_{\mathcal{A}_0}(\alpha) + D_{\mathrm{int}} \qquad (0 \leq \alpha \leq 1). \quad (9)$$

When the interface is loaded lazily so that unused added actions have zero mandatory cost, $D_{\mathrm{int}} = 0$ and the achievable Greppy frontier weakly dominates the lexical frontier.

In plain terms: adding Greppy's commands to an agent's toolbox cannot make an optimal agent worse, because every old action is still available and behaves identically; the only unavoidable cost is the few hundred prompt tokens that describe the new commands. Whether a real, non-optimal model benefits is exactly what the experiment measures.

Proof. For every feasible baseline policy, choose the policy on $\mathcal{A}_1$ that issues exactly the same calls and final answer. Inclusion of the action set and observation equivalence preserve its answer distribution; only the mandatory interface cost can differ. Taking the infimum over all baseline policies proves Eq. (9). □

2.2 Lexical lower bounds

Semantic entry and graph expansion create strict cost gaps on distinct repository families under the following conditions.

2.2.1 Semantic entry under lexical target-blindness

Consider N candidate signatures, exactly one of which implements the target behavior, and draw its index T uniformly. Fix the lexical query class available to the baseline. A repository family is *lexically target-blind* when, conditional on every transcript that has not exposed the target, each permitted query has the same hit-observation distribution for the target as for every uninspected non-target candidate. A grep result containing a candidate-specific byte range or identifying path counts as inspecting that candidate. Paths, filenames, directory structure, documentation, tests, previously rejected bodies, and negative queries are part of the conditioning transcript. Consequently the next uninspected candidate chosen by any policy is independent of T, and the target rank in its inspection order is uniform. A literal or synonym pattern, metadata field, or rejected-source feature that ranks the target violates target-blindness and removes the bound.

Theorem 2 (Lexical entry lower bound)**.** *If the target is uniform over a lexically target-blind set of N candidates, any lexical-search-and-read policy that locates it with probability at least α must inspect at least $\alpha(N+1)/2$ candidate sources in expectation. The bound permits randomized abandonment and is therefore conservative.*

In plain terms: when a behavioural question matches N functions whose names and text look equally plausible, plain grep-and-read must open about half of them on average before it finds the right one. Semantic entry exists to skip that linear scan.

Proof. Under target-blindness, condition on a policy's maximum of $L = \ell$ inspections if no hit occurs. The target's position in its inspection order is uniform, so success is ℓ/N and the expected number actually inspected is $\phi(\ell) = \ell - \ell(\ell-1)/(2N)$. For every $0 \le \ell \le N$, $\phi(\ell) \ge [\ell/N](N+1)/2$. Averaging over randomized L gives $\mathbb{E}[K] \ge \mathbb{P}(\text{success})(N+1)/2$, and the result follows. Equality is attainable by abandoning immediately with probability $1-\alpha$ and otherwise searching all candidates, showing why the stronger αN bound would be false for unrestricted policies. □

Of these inspections, at least $\alpha(N-1)/2$ are rejected candidates in expectation. Let c_{rej} be the source-reading and reasoning cost of rejecting one candidate; the successful candidate belongs to the exact source needed by both methods. Let semantic search have top-k recall ρ_k and let $\nu_k = \mathbb{P}(\text{target retrieved, selected, and source-validated})$ on the same task distribution. If returned signatures cost c_{sig}, the semantic query, LLM selection reasoning, and source validation cost c_{sem}, and the entry index costs B_E to build, semantic entry strictly improves this navigation bound at target success α whenever

$$\nu_k \ge \alpha, \qquad \frac{B_E}{Q_{\text{reuse}}} + c_{\text{sem}} + kc_{\text{sig}} < \frac{\alpha(N-1)}{2} c_{\text{rej}}. \quad (10)$$

Raw recall ρ_k diagnoses the retrieval layer but cannot substitute for ν_k: a target in the list is useless if the LLM does not select and validate it. The threshold is evaluated from N, k, ν_k, and the actual token, index, and latency costs. Literal-name controls should favor the lexical baseline and prevent the task distribution from manufacturing semantic advantage.

2.2.2 Graph expansion under structural ambiguity

Suppose the entry signature v is known. Its unqualified identifier appears at M candidate sites, while only d sites represent the requested typed relation. Here d describes the realized repository and is not revealed to the lexical policy. Overloading, shadowing, imports, and same-name definitions can make the matching lines identical while binding them to different symbols. For occurrence i, a *disambiguating context* D_i is a source slice containing enough lexical scope, import, receiver, and type information for the reference resolver to determine whether that occurrence binds to v.

Theorem 3 (Structural disambiguation lower bound)**.** *There exists a family of repositories with M identical grep-level candidate occurrences such that any policy using only lexical search and source reads that is guaranteed to return the exact neighbor set of v must inspect disambiguating context for all M sites. A correct indexed graph returns the d neighbors in one query with output $\Theta(d)$.*

In plain terms: a text occurrence of a name does not tell the agent whether it is the call it cares about; if M places mention the name, all M must be read to know the true callers. A resolved graph answers the same question in a single query whose size is the answer itself.

Proof. Construct repositories that share every grep observation outside the D_i but independently bind each candidate occurrence either to v or to a same-named definition in a different scope. If a policy leaves one D_i unread, two repositories remain consistent with its transcript but require different answers at that site. The policy cannot be correct on both. Thus all M bindings must be disambiguated. An index that has already resolved the typed edges need only enumerate the d stored neighbors. □

The preceding theorem has a guarantee criterion. A separate distributional bound is needed for target success below one.

Theorem 4 (Success-constrained structural lower bound)**.** *Suppose the M candidate sites bind independently to v with probability $1/2$, every uninspected binding remains independent of the lexical transcript, and inspecting a site reveals its binding. If a lexical-search-and-read policy returns the exact M-bit neighbor vector with probability at least α, then its expected number K of inspected sites satisfies*

$$\mathbb{E}[K] \ge L_G(\alpha) := M\left(\frac{\alpha - 2^{-M}}{1-2^{-M}}\right)_+. \quad (11)$$

The bound permits randomized abandonment and guessing.

In plain terms: if an agent must report exactly which of M look-alike call sites are real, and each site can only be settled by reading it, then high confidence forces reading almost all M sites – no clever ordering escapes this. The typed graph replaces those M reads with one query.

Proof. Conditional on any stopped transcript with $K = k$, the $M - k$ uninspected bits are independent fair bits, so any exact output succeeds with probability at most $2^{-(M-k)}$. For each integer $0 \le k \le M$,

$$2^{k-M} \le 2^{-M} + (1 - 2^{-M})\frac{k}{M}; \tag{12}$$

this is equivalent to $k(2^M - 1) \ge M(2^k - 1)$, which follows because $(2^x - 1)/x$ is increasing for positive integer x. Taking expectations gives $\alpha \le 2^{-M} + (1 - 2^{-M})\mathbb{E}[K]/M$ and hence (11). For $\alpha \ge 2^{-M}$ the lower envelope is attained by mixing an uninspected uniform guess with full inspection, so no stronger bound follows under these assumptions. □

For the guaranteed-correctness family, the d true sites may belong to the exact source ultimately required by both methods; the other $M - d$ inspections are pure navigation overhead. On a complete graph branch, let $\widehat{d}$ be the number of returned candidates, including all d true neighbors, and suppose every one of the $\widehat{d} - d$ false candidates is rejected by local source validation. With per-rejected-site lexical disambiguation cost c_{dis}, graph-query cost c_{graph}, signature output cost c_{sig}, false-candidate validation cost c_{fp}, and graph index build cost B_G, the graph branch is strictly cheaper than the guaranteed lexical branch whenever

$$\frac{B_G}{Q_{\text{reuse}}} + c_{\text{graph}} + \widehat{d}c_{\text{sig}} + (\widehat{d} - d)c_{\text{fp}} < (M - d)c_{\text{dis}}. \tag{13}$$

Equation (13) is explicitly conditional and contains no target-success parameter. Under the independent-binding distribution, let C_G^{tot} instead denote the graph policy's full expected cost, including interface, index, returned false candidates, local validation, exact source reads, and fallback. If its exact-set success rate is $\nu_G \ge \alpha$, let c_{ins} be a lower bound on the complete source-reading and reasoning cost of disambiguating any inspected site, positive or negative. A sufficient success-constrained comparison is

$$\mathbb{E}[C_G^{\text{tot}}] < c_{\text{ins}} L_G(\alpha). \tag{14}$$

This does not substitute the rejection-only cost c_{dis} from (13): $L_G(\alpha)$ counts all inspected sites. If only a lower bound for rejected sites is available, independence and fair bindings yield the weaker expected rejection bound $\frac{1}{2}c_{\text{dis}}L_G(\alpha)$: conditional on deciding to inspect the next site before observing its bit, that site is negative with probability $1/2$, so iterated expectation gives half of $\mathbb{E}[K]$ rejected sites. For the Tree-sitter approximation, missing required edges lower r through G_τ^{cmp}; a false edge accepted despite local evidence contributes to f, while rejected false edges increase $\widehat{d}$ and cost. Missing required Tree-sitter edges are recognized misses when validation triggers lexical fallback and false acceptances otherwise. Rejected false edges increase $\widehat{d}$ and therefore the structured cost. Exact source inspection can validate a returned edge but cannot certify that no required edge was omitted.

2.3 Dominance of the combined Greppy–MSCC policy

The previous two lower bounds concern the two discovery decisions that occur before exact source can be read. We now combine them into a statement about the actual policy the Greppy tool surface induces. Let π_0 be the ordinary lexical-search-and-read coding-agent policy. Let π_{GM} execute the following policy: retain every action of π_0; on an ambiguous task, use semantic entry, traverse typed signature edges, read the selected exact source, and accept the shortcut only after source validation; on a rejected or uncertain shortcut, resume π_0. The coding LLM chooses the calls. Its retrieval, validation, stopping, and fallback errors appear in the probabilities below rather than being assumed away.

Let $\mathcal{D}_H$ be the composed ambiguity family in which (i) the target is uniform among N lexically target-blind signatures and (ii), after entry, each frontier vertex v_i has M_i lexically indistinguishable candidate relation sites, d_i of which are true edges. The task requires exact source for the target and every true neighbor. Candidate bodies used for entry and disambiguating source slices used for expansion are disjoint. By the two preceding theorems, the lexical policy's discovery overhead above the required source is at least

$$L_\tau = \frac{N - 1}{2}c_E + \sum_{i=1}^{m}(M_i - d_i)c_i, \tag{15}$$

where $c_E > 0$ is the cost of rejecting one entry body and $c_i > 0$ the cost of disambiguating one irrelevant relation site. The first term is an expectation over the uniform target; the second is a pointwise guarantee over the indistinguishable binding family. Exact source in the sufficient subgraph is not included in L_τ because both policies must read it.

The construction randomizes the lexical candidate order and ambiguous bindings independently of structured retrieval and validation outcomes. Thus the lower bound in Eq. (15) also holds after conditioning on a structured valid hit. Let this conditional bound be L_H. Equation (15) uses the exact-answer criterion. At a target success level $\alpha < 1$, L_H denotes the corresponding branch-conditioned success-constrained lower bound from the preceding two theorems; the dominance proof below is unchanged.

Let $p = \Pr_{\tau\sim\mathcal{D}}(\tau \in \mathcal{D}_H)$. On the hard family, let r, m, f be the probabilities of a source-validated sufficient hit, a recognized miss followed by lexical fallback, and an insufficient shortcut accepted as sufficient; they sum to one. These joint events include semantic retrieval, Tree-sitter graph coverage, source freshness, generated hint fidelity, and the LLM's decisions. Let S bound expected structured query, returned-metadata, validation-reasoning, and branch-conditional amortized index costs paid before the branch is known. Let K bound the conditional additional transcript cost carried into fallback, and let W bound the conditional additional cost of a false-accept branch relative to the clean lexical run after S has been paid. Let D contain only costs paid regardless of branch, including mandatory interface and instruction tokens and any unconditionally allocated build cost, and let L_0 bound the remaining excess cost on non-hard tasks where π_{GM} uses the lexical path.

Theorem 5 (Conditional no-trade-off: cost and correctness dominance of the combined policy)**.** *Under the definitions*

above,

$$\mathbb{E}[C_{GM} - C_0] \le D + (1-p)L_0 + p\,(S - rL_H + mK + fW)\,. \tag{16}$$

Consequently, π_{GM} is strictly cheaper whenever

$$prL_H > D + (1-p)L_0 + p(S + mK + fW). \tag{17}$$

Let ϵ_0 bound the correctness loss on non-hard tasks, ϵ_h the loss on validated hits, ϵ_m the loss after fallback contamination, and $\ell_f \le 1$ the loss from false acceptance, all relative to π_0 on the same task stratum. Then

$$\mathbb{E}[Y_{GM} - Y_0] \ge -(1-p)\epsilon_0 - p(r\epsilon_h + m\epsilon_m + f\ell_f). \tag{18}$$

Thus π_{GM} is correctness-noninferior within margin δ whenever the positive quantity on the right of Eq. (18) is at most δ.

In plain terms: the combined policy wins whenever the lexical work it avoids (the linear scans and disambiguation reads above, weighted by how often structured evidence exists and validates) outweighs what Greppy itself costs – interface tokens, retrieval calls, misses, and fallbacks. Every term in the inequality is measurable from transcripts; the experiment estimates them instead of assuming them.

Proof. Condition on $\mathcal{D}_H$ and cancel the exact source belonging to the sufficient subgraph, which both policies must read. On a validated hit, π_{GM} pays expected attempt cost S and avoids at least L_H, so the branch contributes at most $S - L_H$. On a recognized miss it pays the structured attempt, then executes the lexical path while retaining the failed attempt in the transcript; the difference is at most $S + K$. On a false acceptance the difference is at most $S + W$. Conditioning on the three branches gives

$$r(S - L_H) + m(S + K) + f(S + W) = S - rL_H + mK + fW,$$

because $r + m + f = 1$. On the complementary task family the difference is at most L_0. Adding mandatory cost D and applying total expectation proves Eq. (16); moving terms across the strict inequality proves Eq. (17).

For correctness, the losses relative to the matched lexical branch are bounded by ϵ_0, ϵ_h, ϵ_m, and ℓ_f on the four disjoint branches. Multiplying each loss bound by its branch probability and applying total expectation proves Eq. (18). □

Corollary 1 (Strict advantage on growing ambiguity classes)**.** *Suppose $p > 0$, the validated-hit probability is bounded below by $r \ge r_{\min} > 0$, false acceptance is zero, validated hits and fallback are correctness-preserving, and D, L_0, S, K are bounded independently of N and the M_i. Then π_{GM} is correctness-noninferior and becomes strictly cheaper as $N + \sum_i (M_i - d_i) \to \infty$.*

In plain terms: the bigger and more ambiguous the codebase – more plausible entry points, more identically-named call sites – the larger Greppy's advantage grows, without bound, while its own costs stay flat. Small, unambiguous repositories are where the two surfaces converge; large production repositories are where they separate.

Proof. The right side of Eq. (17) remains bounded, whereas Eq. (15) and hence prL_H diverges. Equation (18) equals zero under the stated error assumptions. □

This is the promised separation. The graph theorem proves that the exact minimum is intractable in general. The entry and expansion theorems prove a lexical inspection penalty created by two different information losses. The combined theorem proves when the concrete semantic-entry, signature-graph, exact-source, and fallback policy converts those penalties into lower total cost without sacrificing correctness. Tree-sitter omissions, embedding misses, generated-hint errors, and LLM reasoning failures change r, m, f, K, W; they reduce the attainable saving but do not invalidate the bound.

3 Implementation

Greppy exposes the actions in (8) through the commands in Table 1. This correspondence is operational: Table 1 identifies concrete commands, while retrieval success, graph coverage, validation error, latency, and index cost remain measured quantities. Semantic retrieval and signature relations form a single staged navigation interface. For a behavioral description followed by a caller question, representative trajectories are

$$\begin{aligned}
\text{Lexical baseline}: \quad & q \to N \text{ lexical candidates} \\
& \to \text{candidate reads} \to v \\
& \to M \text{ occurrences} \to \{D_i\} \to H_s, \\
\text{Greppy}: \quad & q \to \texttt{semantic-search} \to v \\
& \to \texttt{who-calls / brief} \\
& \to \texttt{expand / --code} \to H_s.
\end{aligned}$$

The LLM chooses the calls; Greppy returns deterministic graph and source evidence plus ranked semantic entry candidates. Local function-purpose hints are navigation aids attached to exact signatures, not evidence that can replace source.

The signature graph is extracted with Tree-sitter and language-specific resolution. Graph-dependent operation is restricted to release-gated languages; reflection, runtime injection, generated code, macro expansion, monkeypatching, and dynamic dispatch can hide edges. Freshness checks reject stale indexed source. Embeddings and summaries are computed locally and reused. Their Metal/CUDA/CPU backend, build time, store size, and generation identifier are recorded so B/Q_{reuse} is not presented as free. The global `--no-summaries` switch disables Qwen hint generation without disabling semantic ranking, graph evidence, or source expansion, making the no-hint ablation executable rather than a post-hoc output filter.

3.1 MSCC-guided navigation

The method factor is the MSCC instruction: a short, tool-independent system-prompt addition, identical on both tool surfaces, that introduces no new operation or hidden state. Before answering or editing, it requires the model to:

Table 1: Formal primitives and current Greppy commands.

Primitive	Concrete operation
Exact baseline	ordinary `grep` arguments, byte-exact passthrough
Semantic entry S_k	`semantic-search`
Name entry	`search-symbols`
Typed neighbors N_r	`who-calls`, `callees`, `find-usages`, `brief`
Transitive/path	`impact`, `path`
Exact source X	`--code`, `expand`
Hint ablation	`--no-summaries` (disables generated hints only; ranking, graph, and expansion stay active)

1. locate the definitions implementing the named or described behavior and read their exact source;
2. follow every caller, callee, usage, type, import, configuration, test, or other dependency that could affect the task, reading the exact source of each relevant node until no relevant relation remains unresolved;
3. use search results, summaries, symbols, and graph relations only to locate source, switching navigation method when a result is incomplete or ambiguous; and
4. exclude code that cannot affect the task, then answer or edit in one pass once the relevant graph is complete.

The instruction names no tool and asks for no cost estimate. Without Greppy, the model must approximate the same task graph through lexical search and source reads; with Greppy, the navigation-surface prompt exposes semantic entry, signature relations, and source materialization. Any additional traversal, reading, or reasoning caused by the method remains in the measured transcript and therefore enters S and K in Theorem 5. The exact MSCC instruction and both tool-surface prompts are reproduced verbatim in Appendix A with their manifest SHA-256 pins.

4 Empirical evidence

Every empirical statement in this paper carries one of four evidence statuses: *completed and analyzed* (all four coding-model panels — MiniMax-M3, GLM-5.2, Qwen3.6-27B, and Kimi-K3 — each run to completion under the frozen plan and the registered prompt set), *release-gated artifact evidence* (the paired edit-and-test benchmark shipped with the evaluated release), *registered* (the mechanism-term estimation for the conditional no-trade-off inequality), and *superseded* (the pilot panel with the earlier prompt set, no longer reported). This section reports all four panels with the frozen automatic task checker.

The four panels share one registered analysis and differ only in the model, so a consistent effect across them separates the tool surface from any single model's idiosyncrasies.

4.1 Factorial experiment

The primary design is shown in Table 3. The lexical-search-and-read surface permits ordinary `grep`, file reads, and standard shell navigation. The Greppy surface adds the commands in Table 1 while retaining the exact baseline actions. The standard instruction asks the model to investigate and answer; the MSCC-guided instruction is the fixed system-prompt addition defined above. Task, repository snapshot, model, decoding settings, verifier, and budgets are paired within each four-cell block.

Table 2: Run accounting for the four completed coding-model panels. All four share the same frozen plan, prompt set, checker, and price snapshot; they differ only in the model and its registered price. All hashes refer to the per-shard run manifests shipped with the artifact.

Task set	115 tasks (96 real + 19 controls), 6 pinned repositories
Prompt set	v2 components (`mscc_skill_v2`), manifest-pinned
Planned cells	6,900 per panel (115 tasks × 4 conditions × 5 budgets × 3 reps
Valid cells	MiniMax-M3 6,811 · GLM-5.2 6,077
	Qwen3.6-27B 4,861 · Kimi-K3 5,274
Excluded cells	provider- or transport-invalid sessions only
Checker	frozen automatic task checker, hash-pinned
Weighting	equal repository weighting per plan
Budgets	1, 2, 4, 8 calls; natural stop (16-call cap)
Prices	frozen 2026-07-14 standard-tier billed rates

Table 3: The 2×2 design separates the navigation tools from the instructions used to operate them.

Tool surface	Standard	MSCC-guided
Lexical search and reads	Lexical	Lexical + instruction
Greppy, `grep`, and file reads	Greppy	Greppy + instruction

The navigation suite contains 96 structural or semantic questions across pinned versions of Serde, Flask, Gson, Zod, Tokio, and Django, plus 19 literal or synthetic controls. For each question, a task-specific checker compares the answer with required files, symbols, and relations. This isolates repository discovery from patch generation and test-suite noise; empirical claims are consequently limited to repository QA and localization, not edit success. Four coding models are prespecified in separate panels: `MiniMax-M3` through the MiniMax API, `z-ai/glm-5.2` and `qwen/qwen3.6-27b` through OpenRouter, and `Kimi-K3` through the Kimi API. Every cell has three independent repetitions at each of the 1-, 2-, 4-, 8-call, and natural-stop budgets. One call is one dispatched tool invocation; every member of a parallel request and every tool error consumes one call, whereas the final answer does not. The model sees its remaining budget and must answer when it is exhausted. Natural stop has a 16-call safety maximum. Infrastructure failures invalidate and rerun the entire cell under the prespecified retry rule rather than granting extra analytic calls. The Pi harness, medium reasoning setting, temperature zero and top-p one where exposed, provider endpoint, returned immutable model identifier, and complete provider response metadata are recorded. A provider that cannot honor a frozen controllable setting is excluded before outcomes are unblinded rather than silently using a mutable default. Repositories, task banks, prompts, tool schemas, parser and resolver rules, embedding and hint checkpoints, ranking parameters, k, source and expansion limits, tool binaries, model IDs, and price snapshots are hashed into each run manifest. Conditions are block-randomized within repository–task–model blocks. A repos-

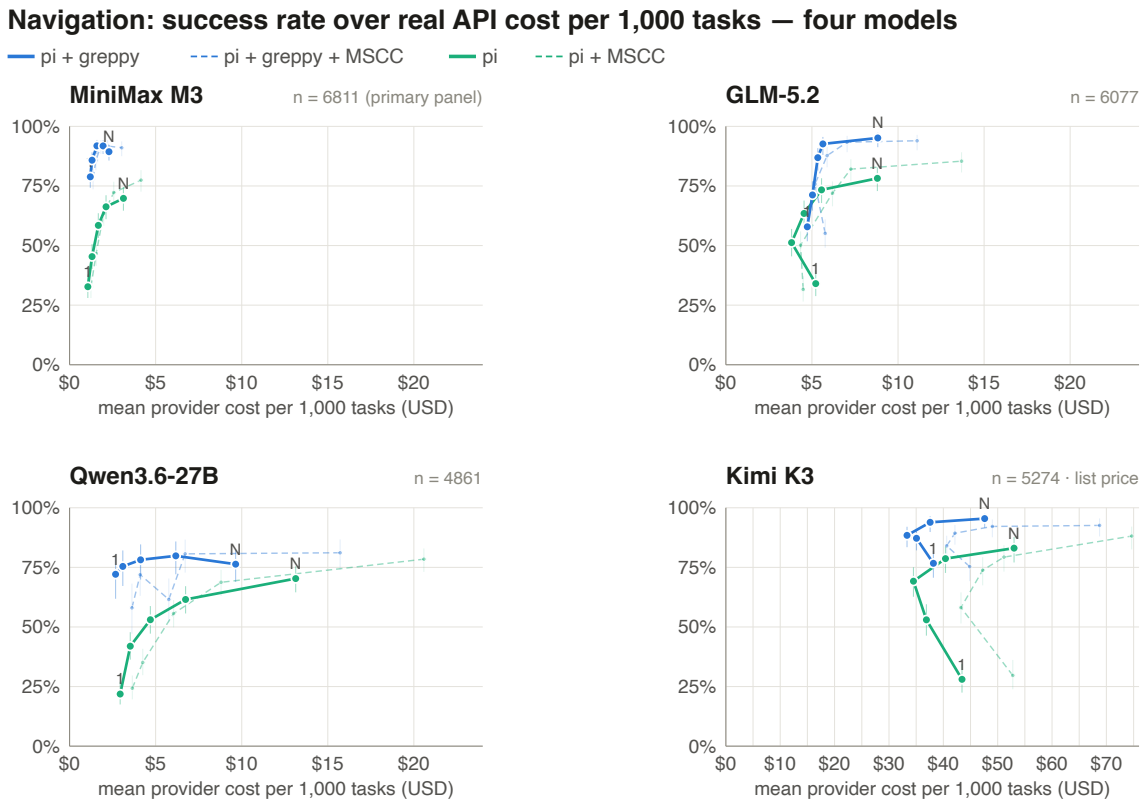

Figure 1: Success rate over mean billed provider cost per 1,000 attempted tasks (linear axis; the three cheaper models share a \$0–24 range, Kimi K3's list-price costs use \$0–76) for the four experimental cells, across all four completed coding-model panels. Each point is one call budget (labelled $1, 2, 4, 8$, N for natural stop) connected in budget order; whiskers are 95% Wilson intervals of the per-condition success rate. In every panel the Greppy surface (blue) lies above and to the left of the lexical baseline (green): more correct at lower cost. The two MSCC-instruction cells (*Lexical + MSCC*, *Greppy + MSCC*; dashed and dimmed) sit essentially on top of their no-instruction counterparts — the tool surface, not the instruction, moves the frontier.

itory index is built once per frozen artifact, its cold build is measured separately, and its immutable warm artifact is shared across conditions and models; no condition can inherit a cache produced by another condition's task transcript.

Before any confirmatory call, a public manifest freezes the exact task list, repository commits, repository and task-class allocation, authoring and exclusion rules, expected structured output, required and forbidden files, symbols, relations, and evidence spans, and the rules used to compute N, M, d, and $\widehat{d}$. Oracles are not generated from Greppy. Static relations are obtained from an available compiler or language-server resolver and independently checked by two annotators; disagreements are adjudicated before tool outputs are examined. For Python, the oracle schema is fixed to lexically resolvable imports, definitions, and direct calls under the pinned environment; dynamic dispatch and monkeypatch behavior are either explicitly annotated from tests or excluded by the frozen task rule, never reclassified after seeing a tool result. Synthetic controls have a hashed generator and a finite grid over candidate count, true degree, source length, and name ambiguity; every grid cell and seed is listed in the manifest rather than selected from pilot outcomes.

The coding-agent panels use external coding models. Greppy's in-house fine-tuned Qwen model instead generates local function-purpose hints inside the navigation tool. It is therefore a tool component affecting U_τ, ranking payload, validation work, and query cost rather than a coding-agent panel. Its checkpoint and prompt hashes are recorded for every Greppy arm. Confirmatory Greppy runs begin only after one checkpoint is selected on disjoint development repositories. The training manifest must exclude the six evaluation repositories, evaluation questions, and derived answers; their hashes and the selection rule are published. Evaluation tasks are never used to choose or replace the checkpoint. A no-hint ablation separates hint quality from graph and embedding effects.

4.2 Mechanism ablations and structured baseline

The factorial study identifies tool-surface and instruction effects, but not which Greppy mechanism causes them. A natural-stop ablation therefore holds the MSCC instruction fixed and compares: (1) grep and file reads; (2) Tree-sitter symbol search with lexical or name entry; (3) semantic entry with exact-source reads but no graph or hints; (4) graph expansion from lexical or name entry with no semantic retrieval or hints; (5) semantic entry plus graph expansion without hints; and (6) full Greppy. Universal Ctags symbol lookup is an additional structured baseline independent of Greppy's semantic and graph layers. Every condition exposes exact source reads and lexical fallback. Tool schemas,

mandatory initialization, index build time, index size, and amortized index cost are measured for each surface. These runs test mechanism attribution; they do not claim superiority over unevaluated LSP- or compiler-based systems. A secondary matched subset uses rust-analyzer for Serde and Tokio, Eclipse JDT LS for Gson, and typescript-language-server with its pinned tsserver for Zod. The public manifest freezes each executable and server version, initialization options, build configuration, supported task list, requested reference or call-hierarchy operation, response normalization, and cold/warm index accounting. A task is excluded from this subset before outcomes are unblinded if the frozen server does not expose the required operation or cannot resolve the pinned build; it remains in the primary lexical comparison. If the subset cannot be completed, all empirical claims remain restricted to the lexical surface and Ctags rather than structured navigation in general.

4.3 Direct measurement of MSCC approximation

Cost and final correctness alone do not show whether a trajectory approached the MSCC. We therefore add a finite oracle-minimal-context suite. Each generated repository contains at most twelve source units with post-training random identifiers and values. The frozen task predicate specifies which source facts are required, the canonical serialization fixes their order, and a context is sufficient exactly when the deterministic task verifier can derive all required facts from the included source units. All 2^n subsets are enumerated before agent runs, so

$$H^*_\tau = \arg\min_{H \subseteq R_\tau} \{\mathrm{tok}(H) : W_\tau(H) = 1\} \tag{19}$$

is computed exactly for every task in this suite rather than estimated from a successful end-to-end trajectory.

For an agent transcript D, the same verifier enumerates all subsets of the exact source records actually returned during discovery. If none is sufficient, the run is a discovery failure. Otherwise let $H^\dagger(D)$ be the smallest sufficient returned subset. The four directly observed outcomes are

$$\begin{aligned} G_{\mathrm{ctx}}(D) &= \mathrm{tok}(H^\dagger(D)) - \mathrm{tok}(H^*_\tau), \\ G_{\mathrm{in}}(D) &= I^{\mathrm{nav}}_D - \mathrm{tok}(H^\dagger(D)), \\ G_{\mathrm{out}}(D) &= O^{\mathrm{nav}}_D, \qquad F_{\mathrm{disc}}(D) = \mathbf{1}\{H^\dagger(D) \text{ undefined}\}. \end{aligned} \tag{20}$$

G_{ctx} measures excess sufficient code, G_{in} measures input spent finding it, G_{out} measures decoded discovery and reasoning output, and F_{disc} records failure to retrieve any sufficient context. The primary MSCC-proximity comparison is Greppy plus the MSCC instruction against the ordinary lexical agent. On larger real repositories, deletion ablation can identify a trajectory-relative $H^\dagger$ but not the global H^*; those results are reported separately and never labelled exact MSCC distance.

4.4 Measurements that test the premises

Every scheduled attempt receives an automatic checker outcome $X \in \{0, 1\}$; timeouts, empty answers, budget exhaustion without a valid answer, and non-infrastructure tool failures receive $X = 0$. Quality is not the unweighted quotient over all attempts: the primary estimand below averages repetitions, then tasks, then gives each repository equal weight. Cost retains the components in (6), tool calls, source opens, returned characters, and wall time. Index construction is reported separately and amortized over deployment scenarios $Q_{\mathrm{reuse}} \in \{1, 10, 100\}$. Discovery and final-answer costs are reported separately and jointly.

The transcript audit splits provider-decoded output at the last tool-calling turn into discovery output and final-answer output. Returned tool input is split into source-bearing and other results; uncached input, cache reads, and cache writes remain separate provider counters. Thus prior decoded output retained in the KV cache is never counted again as new input. This audit does not identify $H^\dagger$ or H^* from end-to-end success alone, so the two latent terms in (5) are not reported as if they were observed. O uses provider-reported output tokens and includes any separately reported reasoning tokens. Unreported internal computation is not estimated; its absence from a provider's telemetry is disclosed. Provider-side history rewrites or compaction, when reported, are recorded as protocol events; when unreported, only billed usage and the visible transcript can be compared.

Each theorem parameter is also measured:

- semantic tasks: chosen k, top-k entry recall ρ_k, validated entry success ν_k, and entry read cost; N is reported only for controlled families in which lexical target-blindness is enforced by construction, not assigned post hoc to real tasks;
- structural tasks: textual occurrence count M, true relation degree d, task indicator g_τ, equal-repository coverage $\gamma_{\mathcal{D}}$, validated neighbor success ν_G, false relations, graph-query cost, c_{ins}, and disambiguating reads;
- evidence reliability: failures e_S, e_G, e_X, e_U, e_A for semantic ranking, graph approximation, source freshness, summary safety, and LLM validation, together with their observed joint rate rather than an independence product;
- hybrid trajectories: shortcut-selection rate η, observable joint consistency $\widehat{r}_{\mathrm{obs}}$, false-accept rate f, residual costs h, w, u, branch-specific matched-baseline costs g_h, g_f, g_m, non-selected effects $d_{\bar{Z}}, e_{\bar{Z}}$, and correctness values $q_h, q_f, q_m, q_{0h}, q_{0f}, q_{0m}$; and
- all tasks: reasoning output required to select, validate, and stop.

The layer events are computed from the complete tool transcript against pinned task oracles. If semantic retrieval is used, S_τ requires its output to contain the target entry marker. If graph expansion is used, G^{cmp}_τ requires coverage of every relation marker in the task oracle; this is an oracle-labelled evaluation event, not something the runtime agent can establish from returned neighbors. X^{snd}_τ requires fresh exact source and local support for every returned relation used by the agent; it does not certify missing-edge absence. U_τ requires either no generated hint or manual confirmation that every audited claim in the returned hints agrees with pinned exact source. A^{orc}_τ is the post-hoc label that the pre-answer stop or fallback decision was appropriate given the hidden oracle events. It is scored without inspecting the final answer or Y and is not described as a runtime observation. A separate adherence field records whether the agent performed the observable source and supported-semantics checks. A layer not invoked by the shortcut is the sure event. Direct source reads

are local soundness validation, whereas a post-shortcut grep, rg, find, or exact-text search is fallback. The observable conjunction is reported as $\widehat{r}_{\text{obs}}$ only after stratified trace review supplies U_τ and checks the automated navigation labels; no independence product and no final-answer label enters it. Final correctness is reported separately within every branch.

Matched-baseline quantities are estimated by first assigning h, f, m from the structured trajectory and then pairing it under Γ with the baseline draw having the same repository, task, model, budget, and prespecified replicate slot. Unless a provider supplies a reliable seed or common-random-number mechanism, this is a matched independent draw, not the unobserved trajectory of the same stochastic run. Thus g_h and q_{0h} describe baseline behavior on tasks whose augmented draws produced a recognized hit; they are not unconditional baseline averages. Runs with $Z = 0$ estimate $d_{\bar{Z}}$ and $e_{\bar{Z}}$ rather than being assumed to cancel.

Answers use a frozen JSON schema with separate file, symbol, relation, evidence-span, and concise-justification fields. The automatic checker scores task success only when all required fields are supported and no asserted relation contradicts the oracle; the positive unit for checker precision and recall is the complete attempted task, while fact-level diagnostics are secondary. A probability sample stratified by model, condition, task class, and checker outcome records each attempt's inclusion probability π_i and is judged independently by two blinded annotators using the frozen rubric. Disagreements go to a third adjudicator, producing $Y_i \in \{0, 1\}$. The audit starts at 240 attempts; before confirmatory outcomes are examined, its final size is the smallest prespecified increment that reaches the joint power target under the frozen arm-specific sensitivity and specificity grid, including differential checker error, or a census if no smaller sample does. The paper reports agreement, attempt-level confusion matrices by arm, and fact-level diagnostics; the uncorrected checker result remains visible.

This instrumentation estimates the observable terms in (10), (13), (14), (17), and (18). The component measurements distinguish semantic or structural savings from lower quality, additional reasoning, or an unamortized index.

4.5 Statistical analysis and central result plot

The primary estimand is defined separately for each coding model on the 96 real tasks at natural stop: Greppy plus the MSCC instruction minus the ordinary lexical-search-and-read agent with its standard instruction. This is the combined-policy comparison proved in Theorem 5. The same-instruction contrast and the two factorial main effects are secondary mechanism comparisons. Let $\mathcal{T}_r$ be the frozen tasks in repository r, let J be the final uniform number of repetitions, let $Y_{cr\tau j}$ denote correctness under the frozen adjudication rubric, and let $C_{cr\tau j}$ be provider dollars. For condition c,

$$Q_c = \frac{1}{6}\sum_{r=1}^{6}\frac{1}{|\mathcal{T}_r|}\sum_{\tau\in\mathcal{T}_r}\frac{1}{J}\sum_{j=1}^{J} Y_{cr\tau j},$$

$$C_c = \frac{1}{6}\sum_{r=1}^{6}\frac{1}{|\mathcal{T}_r|}\sum_{\tau\in\mathcal{T}_r}\frac{1}{J}\sum_{j=1}^{J} C_{cr\tau j}. \tag{21}$$

The bivariate primary contrast is $(\Delta C_{\text{API}}, \Delta Q) = (C_{p-d} - C_{g-a}, Q_{p-d} - Q_{g-a})$. It describes this finite, equally repository-weighted set, not a superpopulation or an unweighted quotient over scheduled attempts.

Because Y_i is manually observed only in the audit sample s_c, quality uses the two-phase Horvitz–Thompson difference estimator. Let X_i be the automatic checker outcome for every scheduled attempt and $w_i = (6|\mathcal{T}_{r(i)}|J)^{-1}$. With frozen inclusion probability π_i,

$$\widehat{Q}_c^{\text{2p}} = \sum_{i\in\mathcal{U}_c} w_i X_i + \sum_{i\in s_c}\frac{w_i}{\pi_i}(Y_i - X_i). \tag{22}$$

The first sum is the complete phase-one frame; the second is the design-unbiased correction for checker error. Arm-specific error is therefore not pooled away.

The primary paired two-phase bootstrap holds the six repositories fixed, resamples tasks and then repetitions within repository while retaining all compared arms, and applies the resulting phase-one multiplicity to both terms of (22). Within every frozen audit stratum it also draws mean-one multinomial replicate weights for the adjudicated records and recomputes the Horvitz–Thompson residual. Thus both trajectory variation and phase-two sampling variation enter each ΔQ replicate. Synthetic controls are resampled separately by generated family and grid cell. Component p-values are obtained by null-recentering these same joint replicates. Correctness non-inferiority and cost superiority use an intersection–union test: the one-sided 95% upper bound for ΔC must be below zero and the one-sided 95% lower bound for ΔQ must be at least $-\delta$, with $\delta = 0.02$. Two percentage points corresponds to approximately two extra errors per 100 scheduled attempts, the largest frozen deployment safety loss; a larger loss is not traded for cost. The maximum component p-value is the model-level intersection–union p-value, and Holm correction controls the family-wise error rate across the prespecified eligible models. The 1-, 2-, 4-, and 8-call comparisons, factorial effects, task-class contrasts, and mechanism ablations are secondary or exploratory and labelled as such.

A frozen simulation uses the fixed repository sizes, paired binary discordance grid, the prespecified differential-checker-error grid, and a blinded cost-dispersion statistic: pooled within-block median absolute deviation of log cost after condition labels are replaced by random codes. No arm mean, arm-specific quality rate, or between-arm difference is available at this decision. The design must show at least 0.80 probability of satisfying both primary bounds under a 20% API-cost reduction and no true quality loss; 20% is the minimum operationally material saving fixed to justify an added index and interface lifecycle. If three repetitions do not meet the criterion, repetitions four through six are added uniformly while the first three remain in the final analysis. Failure at six is reported as insufficient precision, not converted into a positive claim. Models failing a frozen provider-control gate before unblinding are removed before the Holm family is fixed. A model that becomes unavailable afterward remains in that family with no confirmatory rejection and is not replaced. Per-repository effects are always shown. Leave-one-repository-out summaries and a mixed-effects superpopulation model are sensitivity analyses only.

The central plot shows mean billed API cost per 1,000 attempts against correctness, with frozen provider prices on

a linear cost axis. Frozen prices are the provider-billed standard-tier rates at the recorded snapshot date; output-token counts are the provider-billed totals and therefore include reasoning tokens. The harness rejects any cell whose billed context would cross into a higher pricing tier, and all cross-arm ratios within a model are invariant to the absolute price level.

Absolute cost levels are not comparable across models, and deliberately so: billed unit prices do not track parameter count, and caching support differs per serving endpoint. In the measured panels, the 27B Qwen3.6 costs nearly as much per task as the far larger GLM-5.2, for two measured reasons: its billed output rate ($2.40 per million tokens) is 83% of GLM-5.2's despite the size difference – output pricing follows hosting economics, not model scale – and its serving endpoint provides almost no prompt caching (about 3% of its context tokens are served from cache, against 82% for GLM-5.2), so its nominally cheap input rate applies to roughly 5.7× more billed input volume. This is exactly why every confirmatory contrast in this paper is within-model: unit-price structure and caching behaviour cancel between the two arms of the same model, and cross-model cost levels carry no information about surface quality. Separate panels report discovery and final-answer API components. Companion frontiers report warm-index latency and local resource use, and amortized total-cost sensitivities for $Q_{\text{reuse}} \in \{1, 10, 100\}$; benchmark repetition is not treated as deployment reuse. Every point is labelled by budget 1, 2, 4, 8, or N for natural stop. Lines only connect independently executed budgets. In a common paired bootstrap replicate, point x is dominated if another arm or budget for the same model has no greater cost and no lower correctness, with at least one strict inequality.

The formal conditions motivate, but do not imply, the following empirical hypotheses:

- **H1:** Greppy reduces cost on structural tasks with large $M - d$ when graph coverage and LLM validation satisfy (17) without violating (18); zero-degree tasks ($d = 0$) are reported as their own stratum;
- **H2:** Greppy reduces entry reads on behaviorally described tasks when validated entry success ν_k is high, while no advantage is expected on literal-name controls; N from (10) is not imputed to those real tasks; and
- **H3:** the combination of Greppy and the MSCC-guided instruction improves the cost–correctness trade-off when the instruction increases validated shortcut use and timely fallback enough to repay its additional decoded output.

4.6 Measured panels

Each of the four panels contains 6,900 independent sessions: 115 tasks (96 real-repository tasks plus 19 literal and synthetic controls), four conditions, five call budgets, and three repetitions, scored with the frozen automatic checker. Table 4 reports the natural-stop means for all four models; the per-budget frontier is Fig. 1.

The pre-registered primary contrast – the paired natural-stop comparison of Greppy plus the MSCC instruction against the ordinary lexical agent, with a −2-point correctness noninferiority margin – is computed once on each completed panel under the frozen analysis plan and is therefore not reported here. The measured superiority is quantified by task-paired contrasts with cluster-bootstrap 95% intervals

Table 4: Natural-stop operating points for the four completed panels (95% Wilson intervals). In every model the Greppy surface answers more tasks correctly than the lexical baseline at lower or comparable cost; adding the MSCC instruction changes correctness only within the interval width — lowering it slightly on GLM-5.2 and Kimi-K3 — while always adding cost.

Condition	Correct	API cost/task
MiniMax-M3		
Lexical	70% [65, 74]	$0.00313
Lexical + MSCC	77% [73, 82]	$0.00415
Greppy	89% [86, 92]	$0.00229
Greppy + MSCC	91% [88, 94]	$0.00303
GLM-5.2		
Lexical	78% [73, 83]	$0.00880
Lexical + MSCC	85% [81, 89]	$0.01369
Greppy	95% [91, 97]	$0.00882
Greppy + MSCC	94% [90, 96]	$0.01111
Qwen3.6-27B		
Lexical	70% [65, 75]	$0.01313
Lexical + MSCC	78% [73, 83]	$0.02058
Greppy	76% [69, 82]	$0.00964
Greppy + MSCC	81% [74, 87]	$0.01572
Kimi-K3		
Lexical	83% [77, 88]	$0.05301
Lexical + MSCC	88% [83, 92]	$0.07464
Greppy	95% [92, 98]	$0.04760
Greppy + MSCC	93% [88, 96]	$0.06874

(Table 5): on the MiniMax-M3 panel, at every fixed call budget the Greppy surface answers between +33.8 and +44.0 percentage points more tasks correctly than the lexical agent on the same tasks, with every interval bounded away from zero by more than twenty points; at natural stop it is 25.0% cheaper (cost ratio 0.750, interval [0.590, 0.906]) at no lower quality. At natural stop the Greppy surface alone reaches correctness parity with the lexical agent (+10.2 points, interval [−1.7, +23.3]); adding the MSCC instruction (the pre-registered combined policy) yields +16.1 points (interval [+5.3, +28.1]). In the unit that matters operationally – billed dollars per correct answer – the Greppy surface is roughly half as expensive at every operating point: $1.49–$2.39 per 1,000 correct answers against $2.84–$4.17 for the lexical agent. Every Greppy operating point in the MiniMax-M3 panel of Fig. 1 lies above and to the left of every lexical operating point of equal or higher cost.

The budget frontier contains practically relevant measured comparisons that natural-stop means hide. On MiniMax-M3, Greppy at a single tool call already answers 79.0% of tasks correctly for $1.20 per 1,000 tasks — above the lexical agent's best natural-stop operating point (70% for $3.13) at less than half its cost. The same dominance holds in *every* panel: to reach the lexical baseline's best correctness, Greppy costs 62% less on MiniMax-M3, 39% less on GLM-5.2, 80% less on Qwen3.6-27B, and 37% less on Kimi-K3, while answering more tasks correctly at every matched budget (a +6 to +50-point gain). Adding the MSCC instruction moves none of these numbers materially: *Greppy + MSCC* tracks *Greppy* alone to within the interval width in all four panels, and on Kimi-K3 the instruction only adds cost. The

Table 5: Task-paired contrasts, Greppy versus lexical surface (no method instruction on either side), MiniMax-M3 panel (shown as the per-budget exemplar; the other three panels show the same dominance across budgets — Greppy more correct at every fixed budget — in Fig. 1, with natural-stop means in Table 4). Δ success is the within-pair correctness difference in percentage points; the cost ratio divides Greppy by lexical billed provider cost on the same pairs. Brackets are cluster-bootstrap 95% intervals over tasks.

Budget	Pairs	Δ success (pp)	Cost ratio
1	75	+44.0 [+29.2, +57.7]	1.106 [1.030, 1.182]
2	54	+42.6 [+26.5, +58.9]	1.009 [0.904, 1.118]
4	68	+39.7 [+27.7, +52.4]	1.047 [0.966, 1.131]
8	71	+33.8 [+21.7, +46.2]	0.986 [0.868, 1.117]
Natural	59	+10.2 [−1.7, +23.3]	0.750 [0.590, 0.906]

frontier is a property of the tool surface, not of the instruction. These are descriptive cross-budget operating-point comparisons; the confirmatory decision continues to use the paired natural-stop contrast above. The oracle-minimal-context suite in Eq. (19) is analyzed separately because end-to-end cost and correctness do not identify context distance.

4.7 Validity and scope

The action-set bound relies on baseline simulation and a measured upper bound on mandatory interface overhead. Exact weak dominance applies only when that overhead is zero. Byte-identical `grep` output alone does not establish equal cost, and the bound would not apply to a product that replaces `grep` with an incompatible approximation. Greppy's passthrough and no-index behavior are therefore implementation requirements, while schema and dispatch overhead remain measured quantities.

The semantic lower bound describes lexically target-blind task families, not all code search. Literal identifiers can make the lexical baseline optimal. Semantic advantage depends on measured recall, and embeddings can rank plausible but wrong code. The guaranteed graph lower bound is a worst-case ambiguity construction; its success-constrained counterpart additionally assumes independent, uniformly random bindings. Real benefit depends on $M-d$, returned size $\widehat{d}$, and the accuracy of the Tree-sitter-derived edges. Neither proof covers runtime relations absent from static source, and local source inspection does not prove graph completeness.

The factorial experiment isolates Greppy's incremental effect over a lexical-search-and-read coding-agent surface; the ablation adds Ctags and a language-server subset where supported. It does not establish superiority over compiler or LSP indexes outside that matched subset or over the other structured retrieval systems reviewed above; those comparisons require action surfaces and index-cost accounting of their own.

Finally, the optimal-policy frontier is not a guarantee about a particular LLM. A model can waste the larger action set, rely on memorized repository knowledge, or accept an incorrect shortcut. Recent pinned repositories, literal controls, per-model reporting, and the policy factorial reduce but do not remove those threats. Results are limited to the tested models, task distribution, verifier, and cost snapshot.

4.8 Empirical decision

The mathematical result is a no-trade-off statement – formally, dominance: never worse on correctness, cheaper on cost – not a benchmark slogan. Greppy's action set weakly dominates the lexical set when unused extensions have no mandatory cost. On task distributions with semantic or structural ambiguity, the combined Greppy–MSCC policy is strictly cheaper whenever Eq. (17) holds, and it is correctness-noninferior whenever Eq. (18) stays within the chosen margin. Tree-sitter, embedding, generated-hint, and coding-LLM errors enter those two inequalities through measured branch probabilities and costs.

All four panels place the Greppy surface deep in the desired quadrant at every budget, not only at natural stop: the structured surface reaches the lexical agent's best natural-stop correctness already at its smallest call budget for a fraction of the billed cost, and answers more tasks correctly at every matched budget. Per-repository effects point in the same direction, and the advantage is largest exactly where the measured ambiguity moderator predicts it. The MSCC instruction, added to either surface, is a null control in all four panels — the effect is the tool, not the prompt. Registered hypothesis H3 — that adding the MSCC instruction to Greppy improves the cost–correctness trade-off — is therefore not supported: the instruction adds decoded output at no correctness gain, and lowers correctness within the interval on GLM-5.2 and Kimi-K3. Outside this regime the released artifact carries a release-gated paired edit-and-test benchmark on pinned repositories: at exact-McNemar correctness parity the structured arm is billed-cost-noninferior (measured ratio 0.78 against a registered bound of 1.0), so the navigation advantage is not purchased with an edit-regime penalty. The MSCC-distance suite, checker audit, and mechanism measurements extend this evidence under the same frozen design.

Deployment recommendation

The measured evidence supports a direct recommendation: deploy the Greppy surface as the default navigation layer for repository agents. On every measured operating point it answers more tasks correctly per dollar – roughly half the billed cost per correct answer – and at no measured operating point is it worse on either cost or correctness; the paired interval at every fixed budget excludes zero by more than twenty points. On edit tasks outside the navigation regime the release-gated benchmark shows cost- and correctness-noninferiority, so enabling the surface does not tax the workloads it was not built for. The safeguards a cautious integrator would add – exact-source validation before conclusions, lexical fallback on empty or ambiguous structured results, build-and-test verification of edits – are not additional work: they are how the evaluated artifact already operates, and they are the measured configuration, not an untested variant.

References

[1] Anonymous. Greppy: Evidence-bounded code navigation for coding agents. https://github.com/metric-space-ai/greppy, 2026. Open-source software artifact; released version v0.2.0, exact executable hashes for every measured run are pinned in the published run manifests.

[2] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 8697–8727, 2025. doi: 10.18653/v1/2025.acl-long.426.

[3] Wei Cheng, Yuhan Wu, and Wei Hu. Dataflow-guided retrieval augmentation for repository-level code completion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 7957–7977, 2024. doi: 10.18653/v1/2024.acl-long.431.

[4] Hossein Esfandiari, Amin Karbasi, and Vahab Mirrokni. Adaptivity in adaptive submodularity. In *Proceedings of the 34th Conference on Learning Theory*, volume 134 of *Proceedings of Machine Learning Research*, pages 1823–1846, 2021. URL https://proceedings.mlr.press/v134/esfandiari21a.html.

[5] Free Software Foundation. *GNU Grep 3.12 Manual*, 2025. URL https://www.gnu.org/software/grep/manual/html_node/Usage.html. Section 4.16: "What do grep, -E, and -F stand for?".

[6] Daniel Golovin and Andreas Krause. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *Journal of Artificial Intelligence Research*, 42:427–486, 2011. doi: 10.1613/jair.3278.

[7] Yuntong Hu, Tongli Su, Liang Zhao, Bowen Zhu, and Hasibul Haque. LARGER: Lexically anchored repository graph exploration and retrieval, 2026.

[8] Yu Huo, Kun Zeng, Siyu Zhang, Yuquan Lu, Cheng Yang, Yifu Guo, and Xiaoying Tang. RepoShapley: Shapley-enhanced context filtering for repository-level code completion. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 10390–10412, 2026. doi: 10.18653/v1/2026.findings-acl.505.

[9] Haoxiang Jia, Earl T. Barr, and Sergey Mechtaev. Compressing code context for LLM-based issue resolution, 2026.

[10] Hailey Joren, Jianyi Zhang, Chun-Sung Ferng, Da-Cheng Juan, Ankur Taly, and Cyrus Rashtchian. Sufficient context: A new lens on retrieval augmented generation systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2411.06037.

[11] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Springer, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[12] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. ContextBench: A benchmark for context retrieval in coding agents, 2026.

[13] Zhonghang Li, Zongwei Li, Yuxuan Chen, Han Shi, Jiawei Li, Jierun Chen, Haoli Bai, and Chao Huang. FastCode: Fast and cost-efficient code understanding and reasoning, 2026.

[14] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638.

[15] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Qizhe Shieh, and Wenmeng Zhou. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics*, pages 132–151, 2025. doi: 10.18653/v1/2025.naacl-long.7.

[16] Ghassan Misherghi and Zhendong Su. HDD: Hierarchical delta debugging. In *Proceedings of the 28th International Conference on Software Engineering*, pages 142–151, 2006. doi: 10.1145/1134285.1134307.

[17] Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991. doi: 10.1016/0004-3702(91)90015-C.

[18] Chuanqi Shi, Miao Gao, and Zhiqiang Gao. AIRCoder: Adaptive integration of multi-dimensional retrieval for repository-level code completion. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 25458–25470, 2026. doi: 10.18653/v1/2026.acl-long.1166.

[19] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. Perses: Syntax-guided program reduction. In *Proceedings of the 40th International Conference on Software Engineering*, pages 361–371, 2018. doi: 10.1145/3180155.3180236.

[20] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984. doi: 10.1109/TSE.1984.5010248.

[21] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. Repoformer: Selective retrieval for repository-level code completion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 53270–53290, 2024. URL https://proceedings.mlr.press/v235/wu24a.html.

[22] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024.

[23] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, 2023. doi: 10.18653/v1/2023.emnlp-main.151.

[24] Fuwei Zhang, Yanzhao Zhang, Mingxin Li, Dingkun Long, Lexiang Hu, Pengjun Xie, Zhao Zhang, and Fuzhen Zhuang. CORE-Bench: A comprehensive benchmark for code retrieval in the era of agentic coding, 2026.

[25] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. SWE-Explore: Benchmarking how coding agents explore repositories, 2026.

[26] Sheng Zhang, Yifan Ding, Shuquan Lian, Shun Song, and Hui Li. CodeRAG: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 23278–23288, 2025. doi: 10.18653/v1/2025.emnlp-main.1187.

A The MSCC instruction and the Greppy surface prompt, verbatim

The four experimental cells are the bare agent, the agent plus the MSCC instruction, the agent plus Greppy, and the agent plus Greppy plus the MSCC instruction. The two texts below are the method and tool-surface components those cells add; they are byte-identical to the files whose SHA-256 hashes every run manifest pins. The shared discovery preamble, the lexical-surface component, and the neutral method line are pinned the same way and add no further instruction content.

A.1 MSCC instruction

Manifest key `dominance_skill`, SHA-256 `43c936aa5395190f071ba9908598f9261e3b739b7b1dd9b2c5e06131f8a8ac4e`.

```
Before answering or editing, construct the task's source-grounded code context.

1. Locate the definitions that implement the named or described behavior and read their exact source.

2. From these definitions, follow every caller, callee, usage, type, import, configuration, test, or other dependency that could
affect the task. Read the exact source of every relevant node and continue until no relevant relation remains unresolved.

3. Use search results, summaries, symbols, and graph relations only to locate source. If one navigation method is incomplete or
ambiguous, use another; an empty result does not prove that no relation exists.

Exclude code that cannot affect the task. Once the relevant graph is complete, stop exploring and answer or edit in one pass.
```

A.2 Greppy surface prompt

Manifest key `greppy_tools`, SHA-256 `d103e8f389129d8c5ab4880f20d5e920b8f18f680e1efbdc285a32b2b1fbfc9b`.

```
This project has `greppy`, a local code-navigation tool over a symbol graph and
an on-device semantic index. Ordinary grep invocations are delegated byte-for-
byte to the real system grep, but Greppy must not be installed or invoked as a
global grep alias.

CODE-NAVIGATION COMMANDS. SYMBOL is a function / method / class / type name.
They return resolved results as `qualified_name file:line`, not text matches:
  greppy who-calls SYMBOL        the callers of SYMBOL (incoming calls)
  greppy callees SYMBOL          the functions SYMBOL calls (outgoing calls)
  greppy find-usages SYMBOL      every reference to SYMBOL (calls, uses, imports)
  greppy brief SYMBOL            SYMBOL's definition plus its callers and callees, in one call
  greppy impact SYMBOL           the transitive set of code a change to SYMBOL reaches
  greppy search-symbols NAME     definitions whose name matches NAME (a name or fragment)
  greppy path --from A --to B    a call chain from symbol A to symbol B, if one exists

SEMANTIC SEARCH - use when you do NOT know the symbol's name:
  greppy semantic-search "PLAIN-ENGLISH DESCRIPTION"
      Describe the behaviour or code you are looking for in plain English
      (e.g. "restrict a value to a range", "retry a failed HTTP request").
      Returns the closest-matching definitions by meaning (signature + file:line).
      While first-use embeddings are still building, returns a retryable status
      with the active backend, progress, and ETA instead of partial/empty hits.

EXPAND - get the full source in one call instead of opening files by hand:
  greppy expand ID
      who-calls / callees / impact / semantic-search may end their output with a
      line `Expand: greppy expand <id>`. Run it to print the prepared evidence
      pack - the full source of the top matches, bundled - in a single call,
      instead of reading each file:line yourself.

FLAGS (append to any command above):
  --code            include each result's source lines (so no separate read is needed)
  --all             return every result (turn off the default truncation)
  --json            machine-readable output with exact counts
  --root DIR        run against a repo other than the current directory
  --kind KIND       (search-symbols) restrict to function|method|class|struct|enum|trait
  --direction incoming|outgoing, --depth N   (impact) which way and how far to walk
  --from A --to B   (path) the two endpoint symbols

Prefer these over grepping a symbol name and reading every hit: who-calls /
callees / impact answer relationship questions directly, and semantic-search
finds code you cannot name.

Treat returned source paths, exact spans, signatures, and graph relations as
evidence. The indented English sentence below a function signature is a local
Qwen navigation hint. Read the source and verify changes with builds and tests.
```